

Plan Técnico de Sprints Semanales (Meticuloso y Completo) - Centralización de Procesos

Cliente: Fundación Infantil Santiago Corazón

Equipo de Desarrollo: Backlog Técnico Operativo

- **Duración del Proyecto:** 5 Meses (20 Semanas)
- **Estructura de Sprints:** 20 Sprints de 1 semana cada uno.
- **Stack:** Angular (v16+) | Node.js (NestJS / TypeScript) o Python (FastAPI) | PostgreSQL Dedicado (v15+).

Este documento contiene la planificación detallada y exhaustiva, diseñada para garantizar el cumplimiento al 100% del alcance funcional y técnico de la **Propuesta 2** y las plataformas identificadas en el **Inventario de Plataformas y Accesos.xlsx**.

1. Mapeo de Requerimientos del Contrato vs. Backlog de Sprints

Requerimiento del Contrato / Propuesta 2	Ubicación en el Backlog Técnico
Módulo CRM (Clientes y Donantes)	Sprint 5 (Tareas 5.1 - 5.5) y Sprint 6 (Tarea 6.4)
Migración de Historial Access	Sprint 6 (Tareas 6.1 - 6.3)
Módulo de Pedidos Personalizados (Manuales)	Sprint 7 (Tareas 7.1, 7.2) y Sprint 8 (Tareas 8.1 - 8.4)
Integración con Shopify Webhooks	Sprint 9 (Tareas 9.1 - 9.3) y Sprint 10 (Tareas 10.1 - 10.3)
Integración de Pasarelas (Wompi, PayU, PayPal, Frecuenti)	Sprint 11 (Tareas 11.1 - 11.4)
Generación de Certificados Automáticos en PDF	Sprint 12 (Tareas 12.1 - 12.3)
Servicio de Envío Automático SMTP (Email)	Sprint 13 (Tareas 13.1 - 13.3)
Módulo de Inventarios (Productos, Insumos, Acompañantes)	Sprint 14 (Tareas 14.1 - 14.3) y Sprint 15 (Tareas 15.1 - 15.3)
Lógica de Descuento Automático de Stock (Recetas/Insumos)	Sprint 16 (Tareas 16.1 - 16.3)
Integración con WhatsApp Business API	Sprint 17 (Tareas 17.1 - 17.3)
Dashboard Privado (Métricas Financieras y de Gestión)	Sprint 18 (Tareas 18.1 - 18.6)
Dashboard Público y Mapa Geográfico	Sprint 19 (Tareas 19.1 - 19.4)
Exportación Contable (World Office)	Sprint 20 (Tarea 20.1)

Roles, Permisos y Autenticación JWT (Admin, Director, Tienda, Factura)	Sprint 3 (Tareas 3.1 - 3.5) y Sprint 4 (Tareas 4.1 - 4.5)
DevOps, Docker, SSL y Backups	Sprints 1, 2 y Sprint 20 (Tareas 20.2, 20.3)

2. Plan Técnico Semana a Semana (20 Sprints)

SPRINT 1 (Semana 1): Infraestructura de Servidor y PostgreSQL Dedicado

- **Tarea 1.1 [DevOps] - Aprovisionamiento y Hardening de PostgreSQL**
 - *Descripción:* Instalar y configurar PostgreSQL 15+ en el servidor dedicado/VPS. Configurar el puerto `5432` con encriptación SSL en tránsito (`force_ssl=on`) y restringir accesos mediante archivo `pg_hba.conf` para aceptar conexiones únicamente desde las IPs del backend y del desarrollador.
 - *DoD:* Conexión exitosa desde PGAdmin utilizando túnel SSL verificado.
- **Tarea 1.2 [Backend] - Configuración de Migraciones ORM**
 - *Descripción:* Inicializar Prisma (Node.js) o Alembic (Python) en el backend. Configurar variables de entorno `.env` con la cadena de conexión cifrada.
 - *DoD:* Migración de prueba (`init`) ejecutada y guardada en base de datos.
- **Tarea 1.3 [Backend] - Definición del Esquema de Roles y Permisos**
 - *Descripción:* Crear migraciones para las tablas relacionales `roles` , `permissions` y `role_permissions` en PostgreSQL. Escribir un script de seed conteniendo los roles: `Administrador` , `Director` , `Operador Tienda` , y `Facturador` .
 - *DoD:* Tablas existentes en BD y script de seed ejecutado cargando los roles estándar.
- **Tarea 1.4 [Frontend] - Inicialización de Angular Corporativo**
 - *Descripción:* Generar el proyecto Angular (`ng new`) configurando SCSS y enrutamiento modular. Integrar el framework visual (Tailwind CSS o Angular Material).
 - *DoD:* Ejecución de `ng build` exitosa sin errores de compilación.
- **Tarea 1.5 [Frontend] - Setup de Ambientes y Variables Globales**
 - *Descripción:* Configurar variables del backend (`API_URL` , `TIMEOUT`) en los archivos `environment.ts` y `environment.prod.ts` .
 - *DoD:* Variables cargadas dinámicamente según la compilación elegida.

SPRINT 2 (Semana 2): Boilerplate API, Dockerización y Capa HTTP Angular

- **Tarea 2.1 [Backend] - Estructuración del Boilerplate Backend**
 - *Descripción:* Diseñar e implementar la estructura del API backend basada en controladores (Controllers), servicios (Services) y repositorios (Repositories).
 - *DoD:* El endpoint `/api/health` retorna estado `UP` en formato JSON.
- **Tarea 2.2 [Backend] - Middleware Global de Excepciones y Logs de Auditoría**
 - *Descripción:* Crear un manejador global de excepciones para interceptar errores de servidor y base de datos, mapeándolos en la tabla `audit_logs` (id, user_id, action, ip_address, timestamp).
 - *DoD:* Respuestas fallidas retornan código HTTP controlado en vez de stack traces expuestos.
- **Tarea 2.3 [DevOps] - Dockerización de Desarrollo Local**
 - *Descripción:* Configurar `Dockerfile` para la API backend y un archivo `docker-compose.yml` que levante el contenedor de PostgreSQL y el backend sincronizados en una red Docker interna.

- *DoD*: Ejecutar `docker-compose up` levanta el ambiente completo local de forma autónoma.
 - **Tarea 2.4 [Frontend] - Arquitectura Angular Core y Shared**
 - *Descripción*: Crear los directorios `core/` (servicios globales, interceptores), `shared/` (componentes comunes), y `modules/` (módulos funcionales lazy-loaded).
 - *DoD*: Estructura de carpetas libre de importaciones circulares en el código.
 - **Tarea 2.5 [Frontend] - Interceptor HTTP de Notificaciones**
 - *Descripción*: Programar un interceptor HTTP en Angular que intercepte errores de backend (400, 500, etc.) y muestre automáticamente un mensaje Toast (alerta visual) en la pantalla del usuario.
 - *DoD*: Respuestas fallidas de la API disparan la alerta en el navegador.
-

17 SPRINT 3 (Semana 3): Base de Datos de Usuarios y Autenticación JWT

- **Tarea 3.1 [Backend] - Esquema SQL de Usuarios**
 - *Descripción*: Crear e implementar la tabla `users` (`id`, `email`, `password_hash`, `first_name`, `last_name`, `role_id`, `is_active`, `created_at`, `updated_at`).
 - *DoD*: Llave foránea de `role_id` apuntando correctamente a la tabla `roles`.
 - **Tarea 3.2 [Backend] - Endpoint de Registro Seguro (Bcrypt)**
 - *Descripción*: Implementar el endpoint `POST /api/auth/register` (privado para Administradores). Cifrar las contraseñas usando la librería `bcrypt` con un salt de 10 iteraciones.
 - *DoD*: Guardar un usuario almacena la contraseña de forma irreversible en PostgreSQL.
 - **Tarea 3.3 [Backend] - Endpoint de Login y Firma JWT**
 - *Descripción*: Programar el endpoint `POST /api/auth/login`. Al autenticar con éxito, firma y retorna un token JWT que contenga el `userId` y el `role` del usuario (expiración: 8 horas).
 - *DoD*: Inicio de sesión correcto retorna token JWT y datos de perfil en formato JSON.
 - **Tarea 3.4 [Frontend] - Servicio Angular de Autenticación**
 - *Descripción*: Implementar `AuthService` en Angular para consumir la API de login, almacenar el JWT en `localStorage`, y manejar el estado de sesión activa mediante un `BehaviorSubject`.
 - *DoD*: Almacenamiento correcto del token JWT verificado en el navegador del usuario.
 - **Tarea 3.5 [Frontend] - Formulario de Login Reactivo**
 - *Descripción*: Desarrollar la vista de Login en Angular con validaciones en tiempo real para correos y caracteres de contraseña.
 - *DoD*: Formulario bloquea el botón de envío si los campos son inválidos.
-

17 SPRINT 4 (Semana 4): Roles, Permisos y Layout Administrativo

- **Tarea 4.1 [Frontend] - Guardias de Rutas (AuthGuard y RoleGuard)**
 - *Descripción*: Crear guardias en Angular para proteger el acceso a rutas. Si no hay token JWT, redirige a `/login`. Si el rol del token no cuenta con los permisos de la ruta, bloquea el acceso.
 - *DoD*: Navegar a `/admin/config` con un rol no autorizado retorna pantalla de acceso denegado.
- **Tarea 4.2 [Frontend] - Layout Administrativo Responsive**

- *Descripción:* Programar el cascarón visual del panel administrativo (menú lateral colapsable, barra superior de perfil, y contenedor dinámico para rutas hijas).
- *DoD:* Diseño adaptable a smartphones y laptops sin deformaciones.
- **Tarea 4.3 [Frontend] - Navegación Adaptativa según Permisos**
 - *Descripción:* Decodificar los permisos del token JWT y ocultar/mostrar elementos del menú lateral del Layout basándose en ellos (RBAC).
 - *DoD:* El menú de un usuario con rol de "Facturador" no muestra opciones administrativas.
- **Tarea 4.4 [Backend] - API para Administración de Usuarios**
 - *Descripción:* Desarrollar los endpoints de administración: GET /api/users (paginado) y PUT /api/users/:id (para activar, desactivar o cambiar de rol a usuarios).
 - *DoD:* APIs bloquean peticiones que no vengan de un token JWT con rol "Administrador".
- **Tarea 4.5 [Frontend] - Interceptor para Inyección de Token**
 - *Descripción:* Configurar un interceptor HTTP en Angular que agregue de forma automática la cabecera Authorization: Bearer <token> en todas las peticiones a la API.
 - *DoD:* El backend recibe e identifica el token JWT en las cabeceras HTTP de Angular.

SPRINT 5 (Semana 5): Módulo CRM (Clientes y Donantes)

- **Tarea 5.1 [Backend] - Esquema SQL del CRM**
 - *Descripción:* Crear la tabla clients_donors (id, name, doc_type, doc_number, phone, email, address, city, commune, neighborhood, status [Activo, Inactivo], metadata).
 - *DoD:* Índices creados para búsquedas eficientes en las columnas doc_number y email.
- **Tarea 5.2 [Backend] - Endpoints CRUD del CRM**
 - *Descripción:* Desarrollar endpoints GET /api/crm/clients (filtrado por documento, nombre o tipo de contacto), POST /api/crm/clients, y PUT /api/crm/clients/:id.
 - *DoD:* Endpoints responden exitosamente con datos del cliente y manejan de forma segura colisiones de registros duplicados por identificación.
- **Tarea 5.3 [Frontend] - Estructura Angular del CRM**
 - *Descripción:* Generar el módulo Lazy-loaded crm y configurar rutas correspondientes para el listado e ingreso de clientes.
 - *DoD:* Módulo registrado y accesible desde la barra de navegación lateral.
- **Tarea 5.4 [Frontend] - Tabla CRM con Paginación Servidor**
 - *Descripción:* Programar vista en Angular para listar clientes con buscador interactivo y control de paginación integrada con los parámetros de la API backend.
 - *DoD:* Filtrado de clientes por texto busca en el servidor y actualiza la lista.
- **Tarea 5.5 [Frontend] - Formulario de Creación de Clientes**
 - *Descripción:* Diseñar el formulario reactivo de registro de cliente con validaciones de campos telefónicos, cédula y correo.
 - *DoD:* Guardar datos crea exitosamente el registro en PostgreSQL y refresca la tabla del CRM.

SPRINT 6 (Semana 6): Migración e Integridad de Access a PostgreSQL

- **Tarea 6.1 [Data/Backend] - Script de Extracción y Limpieza ETL**
 - *Descripción:* Escribir script en Node/Python que lea la base de datos Access (.mdb / .accdb), analice y corrija registros inválidos (emails mal formateados, teléfonos inconsistentes).
 - *DoD:* Limpieza automática de datos nulos y formato unificado de números telefónicos.
- **Tarea 6.2 [Data/Backend] - Carga y Mapeo en Base de Datos Postgres**

- *Descripción:* Ejecutar script ETL para insertar masivamente los registros históricos en la tabla `clients_donors`, mapeando el ID de Access en la columna `historical_id`.
 - *DoD:* Registros históricos cargados e indexados correctamente en PostgreSQL.
 - **Tarea 6.3 [Backend] - Log de Auditoría de la Migración**
 - *Descripción:* Guardar logs detallados del proceso de migración conteniendo el total de registros exitosos e inválidos descartados.
 - *DoD:* Reporte de auditoría de migración generado y archivado.
 - **Tarea 6.4 [Frontend] - Vista de Perfil y Detalle del Cliente**
 - *Descripción:* Diseñar la pantalla de ficha de cliente en Angular que muestre su historial de compras y donaciones, así como sus datos históricos de la migración.
 - *DoD:* Ficha del cliente carga correctamente los datos de los usuarios migrados.
-

17 **SPRINT 7 (Semana 7): Modelado de Pedidos y Transacciones ACID**

- **Tarea 7.1 [Backend] - Migración SQL de Pedidos**
 - *Descripción:* Crear migraciones para las tablas `orders` y `order_items`.
 - `orders` : id, client_id, order_date, status, total_amount, payment_status, source, created_by.
 - `order_items` : id, order_id, product_id, quantity, unit_price, subtotal.
 - *DoD:* Integridad referencial configurada con borrado restringido (`ON DELETE RESTRICT`).
 - **Tarea 7.2 [Backend] - Lógica Transaccional ACID para Pedidos**
 - *Descripción:* Programar el servicio de creación de pedidos en el backend de forma que la orden principal y sus líneas de detalle (`order_items`) se inserten de forma atómica en una transacción.
 - *DoD:* Si ocurre un fallo al insertar un producto inválido, la base de datos realiza un rollback completo y no almacena ningún registro huérfano.
 - **Tarea 7.3 [Frontend] - Módulo Angular de Pedidos**
 - *Descripción:* Configurar el módulo Lazy-loaded `orders` y definir rutas para el listado general e ingreso manual.
 - *DoD:* Módulo registrado y accesible desde la barra lateral.
-

17 **SPRINT 8 (Semana 8): Interfaz Angular para Pedidos Manuales**

- **Tarea 8.1 [Frontend] - Formulario Dinámico de Pedido con Autocompletado**
 - *Descripción:* Desarrollar el formulario en Angular con un buscador autocompletado del CRM. Al seleccionar el cliente, carga sus datos de dirección de forma automática.
 - *DoD:* Escribir en el buscador filtra clientes existentes y permite seleccionarlos con un clic.
- **Tarea 8.2 [Frontend] - Selector Dinámico de Productos y Cálculo de Totales**
 - *Descripción:* Implementar una grilla dinámica en el formulario para agregar productos de un selector, cambiar cantidades numéricas y calcular subtotales y total general de forma reactiva.
 - *Criterio de Aceptación:* Modificar la cantidad de un ítem recalcula el total a pagar instantáneamente en pantalla.
- **Tarea 8.3 [Backend] - Endpoint de Consulta e Historial de Pedidos**
 - *Descripción:* Desarrollar `GET /api/orders` con filtros por fecha de creación, origen del pedido (Shopify, Manual, TiendaFísica) y estado de entrega.
 - *Criterio de Aceptación:* API retorna el JSON paginado de pedidos de manera correcta.
- **Tarea 8.4 [Frontend] - Listado y Gestión de Estados del Pedido**

- *Descripción:* Diseñar el listado general de pedidos en Angular, incluyendo una columna interactiva para actualizar el estado del pedido (Recibido, En preparación, Despachado, Entregado).
- *Criterio de Aceptación:* Cambiar el estado de un pedido desde la tabla de Angular actualiza el registro en la base de datos y refresca la vista.

SPRINT 9 (Semana 9): Receptor de Webhooks de Shopify y Seguridad

- **Tarea 9.1 [Backend] - Endpoint del Webhook de Shopify**
 - *Descripción:* Desarrollar el endpoint público `POST /api/webhooks/shopify/orders` listo para recibir notificaciones HTTP POST de Shopify.
 - *Criterio de Aceptación:* El endpoint responde con estado 200 al recibir peticiones HTTP básicas.
- **Tarea 9.2 [Backend] - Validación de Firma Criptográfica HMAC**
 - *Descripción:* Implementar la verificación de firma digital de Shopify. Comparar el header `X-Shopify-Hmac-SHA256` con el hash calculado del cuerpo crudo del request utilizando la clave secreta compartida en Shopify.
 - *Criterio de Aceptación:* Peticiones sin firma válida o con firma alterada reciben respuesta 401 Unauthorized y son bloqueadas.
- **Tarea 9.3 [Backend] - Tabla de Auditoría de Webhooks**
 - *Descripción:* Crear la tabla `shopify_sync_logs` (id, payload, status [Success, Error], error_message, timestamp).
 - *Criterio de Aceptación:* Cada llamada al Webhook registra una fila en la base de datos con su estado de procesamiento.

SPRINT 10 (Semana 10): Procesamiento e Integración de Pedidos Shopify

- **Tarea 10.1 [Backend] - Mapeo de Payload de Shopify a Base de Datos**
 - *Descripción:* Desarrollar servicio para procesar el JSON de Shopify. Si el cliente no existe por correo, crearlo en `clients_donors`. Luego, mapear la orden y sus productos correspondientes.
 - *Criterio de Aceptación:* El pedido de Shopify se inserta en las tablas locales `orders` y `order_items` con origen `Shopify`.
- **Tarea 10.2 [Backend] - Homologación de Productos de Shopify**
 - *Descripción:* Programar la conversión entre el SKU/ID de Shopify de los productos comprados y los IDs internos del catálogo local de la base de datos.
 - *Criterio de Aceptación:* Los productos de Shopify se mapean a los productos de base de datos local de forma exacta.
- **Tarea 10.3 [Frontend] - Consola de Logs de Sincronización en Angular**
 - *Descripción:* Diseñar una pantalla en Angular para visualizar la tabla `shopify_sync_logs` para auditar errores de sincronización y disparar reintentos de Webhook si falla la red.
 - *Criterio de Aceptación:* Vista visualiza logs de Shopify correctamente con códigos de colores (verde para éxito, rojo para error).

SPRINT 11 (Semana 11): Captura de Donaciones de Pasarelas de Pago (Wompi, PayU, PayPal, Frecuenti)

- **Tarea 11.1 [Backend] - Estructura SQL de Donaciones**
 - *Descripción:* Crear e implementar la tabla `donations` (id, client_id, amount, date, payment_gateway, transaction_id, status, campaign, concept, created_at).

- *Criterio de Aceptación:* Migración PostgreSQL aplicada correctamente.
 - **Tarea 11.2 [Backend] - Endpoint de Captura de Webhook de Wompi**
 - *Descripción:* Desarrollar `POST /api/webhooks/wompi` para capturar el cambio de estado de transacciones de donación en línea.
 - *Criterio de Aceptación:* Transacciones con estado `APPROVED` se registran automáticamente en la tabla `donations`. Peticiones duplicadas por reintentos de red se descartan de forma segura.
 - **Tarea 11.3 [Backend] - Webhooks de PayU y PayPal**
 - *Descripción:* Desarrollar receptores y parseadores para las notificaciones de PayU y PayPal de forma análoga a Wompi.
 - *Criterio de Aceptación:* Registros de pagos exitosos de donaciones ingresan a base de datos de forma segura.
 - **Tarea 11.4 [Frontend] - Módulo Angular de Donaciones**
 - *Descripción:* Crear pantalla en Angular para listar las donaciones aprobadas en formato de tabla, con filtros por pasarela, campaña y rango de fechas.
 - *Criterio de Aceptación:* Listado interactivo en Angular muestra las donaciones reales en la base de datos PostgreSQL.
-

SPRINT 12 (Semana 12): Motor de Certificados de Donación PDF

- **Tarea 12.1 [Backend] - Tabla y Consecutivos de Certificados**
 - *Descripción:* Diseñar la tabla `certificates` (`id`, `donation_id`, `certificate_number`, `issue_date`, `pdf_path`, `status`). Configurar el número consecutivo para que sea único e incremental de forma segura.
 - *Criterio de Aceptación:* Llave única de consecutivo contable integrada en PostgreSQL.
 - **Tarea 12.2 [Backend] - Motor de Generación de PDF en el Backend**
 - *Descripción:* Programar un servicio en el backend usando una librería como PDFKit para compilar una plantilla PDF con el logo oficial de la Fundación, firma autorizada, datos del donante, valor monetario (letras y números), y consecutivo contable único.
 - *Criterio de Aceptación:* Generación de PDF exitosa guardándose localmente en el servidor.
 - **Tarea 12.3 [Frontend] - Botón de Descarga Directa en Angular**
 - *Descripción:* Diseñar una interfaz interactiva dentro de la grilla de certificados en Angular para permitir la descarga directa del archivo PDF generado.
 - *Criterio de Aceptación:* Clic en el botón descarga el PDF con el nombre formateado (`certificado_consecutivo.pdf`) al computador del usuario.
-

SPRINT 13 (Semana 13): Automatización de Correos SMTP para Donantes

- **Tarea 13.1 [Backend] - Integración de Servicio de Correo SMTP**
 - *Descripción:* Configurar la librería Nodemailer en backend apuntando a las credenciales de correo SMTP de la fundación.
 - *Criterio de Aceptación:* Correo de prueba se envía y recibe de forma correcta con adjuntos.
- **Tarea 13.2 [Backend] - Disparador de Envío Automático al Donar**
 - *Descripción:* Desarrollar el evento en el backend: tan pronto el webhook de una pasarela confirma un pago exitoso y se genera el PDF, el sistema dispara el envío del correo de agradecimiento con el PDF adjunto de manera automática.
 - *Criterio de Aceptación:* Una simulación de webhook de Wompi exitosa culmina con la recepción automática del correo en la bandeja de entrada del donante simulado.

- **Tarea 13.3 [Frontend] - Historial de Envíos y Reenvío Manual**

- *Descripción:* Diseñar en la interfaz de Angular una columna que muestre si el certificado fue enviado con éxito por correo y un botón para forzar un reenvío manual inmediato si es necesario.
- *Criterio de Aceptación:* Clic en "Reenviar" ejecuta una petición a la API y notifica visualmente del éxito del reenvío del correo.

SPRINT 14 (Semana 14): Catálogo de Inventario, Insumos y Recetas

- **Tarea 14.1 [Backend] - Esquema SQL de Inventarios**

- *Descripción:* Diseñar las tablas en PostgreSQL para el control de stock: `products`, `inputs` (materias primas), `companions` (bolsas, moños, envolturas), y `product_inputs` (tabla que define cuántos gramos/unidades de cada insumo requiere un producto terminado).
- *Criterio de Aceptación:* Tablas creadas e integradas relacionalmente con integridad referencial activa.

- **Tarea 14.2 [Backend] - CRUD del Catálogo de Inventario**

- *Descripción:* Escribir endpoints para administrar el inventario físico: `GET /api/inventory/items`, `POST /api/inventory/items` y `PUT /api/inventory/items/:id`.
- *Criterio de Aceptación:* API responde con el listado de productos, insumos y acompañantes.

- **Tarea 14.3 [Frontend] - Módulo Angular de Inventarios y Tablas**

- *Descripción:* Diseñar en Angular la tabla visual de productos, materias primas y acompañantes con sus SKU, existencias y estados.
- *Criterio de Aceptación:* Interfaz muestra las listas de elementos reales en la base de datos de manera correcta.

SPRINT 15 (Semana 15): Ajustes Manuales y Alertas de Stock Mínimo

- **Tarea 15.1 [Backend] - Tabla y Lógica de Movimientos de Inventario**

- *Descripción:* Crear la tabla `inventory_movements` (`id`, `item_type`, `item_id`, `movement_type` [Entrada, Salida, Ajuste], `quantity`, `user_id`, `description`, `timestamp`).
- *Criterio de Aceptación:* Cada ajuste manual de stock realizado por un usuario registra un log de auditoría detallado en la tabla.

- **Tarea 15.2 [Frontend] - Modal para Registrar Ajuste Manual de Inventario**

- *Descripción:* Desarrollar un formulario modal interactivo en Angular que permita al operador seleccionar un insumo o producto, ingresar una cantidad y registrar un tipo de ajuste (ej: pérdida por rotura).
- *Criterio de Aceptación:* Enviar el formulario descuenta o suma el stock en base de datos y recarga la tabla en la interfaz.

- **Tarea 15.3 [Frontend] - Sistema de Alertas Visuales por Stock Mínimo**

- *Descripción:* Programar la tabla de Angular para que compare la columna `stock` frente a `min_stock`. Si el valor es menor, colorea la fila en rojo e incluye un icono de alerta visual.
- *Criterio de Aceptación:* Productos o insumos con stock crítico resaltan de forma llamativa al cargar el panel de inventario.

SPRINT 16 (Semana 16): Descuento Automático de Stock (Lógica Transaccional)

- **Tarea 16.1 [Backend] - Servicio Transaccional de Descuento de Stock**

- *Descripción:* Programar a nivel de backend: cuando el estado de un pedido (manual o de Shopify) cambie a "En preparación", el sistema debe ejecutar una transacción SQL para restar de forma automática el stock del producto de la tabla `products` y sus correspondientes insumos de las tablas `inputs` y `companions` basado en sus recetas.
 - *Criterio de Aceptación:* Una compra de una pulsera descuenta una unidad del stock de la pulsera, más los insumos definidos en su receta (hilo, dije, bolsa de empaque).
 - **Tarea 16.2 [Backend] - Prevención de Condiciones de Carrera (Race Conditions)**
 - *Descripción:* Implementar sentencias de bloqueo de filas (`SELECT ... FOR UPDATE` en base de datos) durante las transacciones de actualización de stock para evitar lecturas sucias en compras simultáneas.
 - *Criterio de Aceptación:* Peticiones simultáneas concurrentes descuentan de forma consecutiva y exacta sin generar stock negativo.
 - **Tarea 16.3 [Frontend] - Validaciones Visuales de Falta de Stock**
 - *Descripción:* Configurar la vista de despacho en Angular para mostrar advertencias si no hay existencias físicas suficientes para procesar un pedido entrante.
 - *Criterio de Aceptación:* Botón de despacho muestra advertencia y pide confirmación manual de ajuste de stock si no hay mercancía suficiente.
-

SPRINT 17 (Semana 17): Notificaciones por WhatsApp Business API

- **Tarea 17.1 [Backend] - Integración del SDK/Client de WhatsApp API**
 - *Descripción:* Programar en backend la integración para realizar llamadas HTTP POST seguras a la API de WhatsApp Cloud (o proveedor Twilio) con tokens de portador permanentes.
 - *Criterio de Aceptación:* Envío exitoso de mensaje básico de prueba a número del desarrollador.
 - **Tarea 17.2 [Backend] - Disparador de Alertas por Cambio de Estado de Pedidos**
 - *Descripción:* Configurar la llamada automática al servicio de WhatsApp cuando el estado del pedido pase a "Despachado", enviando un mensaje con los datos de transporte al teléfono del cliente.
 - *Criterio de Aceptación:* Cambiar el estado de un pedido en el panel de Angular envía la plantilla de WhatsApp correspondiente de forma inmediata.
 - **Tarea 17.3 [Backend] - WhatsApp para Donantes con Enlace de Certificado**
 - *Descripción:* Programar el envío automático del mensaje de WhatsApp de agradecimiento al donante cuando el webhook aprueba su pago, incluyendo en el mensaje un enlace seguro de descarga directa del certificado PDF.
 - *Criterio de Aceptación:* Una donación exitosa envía un mensaje que contiene el enlace al PDF del certificado de donación.
-

SPRINT 18 (Semana 18): Dashboard Privado y Métricas de Gestión

- **Tarea 18.1 [Backend] - Queries SQL Agregadas para Ventas y Donaciones**
 - *Descripción:* Desarrollar las consultas SQL que calculen las ventas agrupadas por rangos temporales (día, mes, trimestre, semestre, año) e ingresos de donaciones segmentados por campaña activa y pasarela de pago.
 - *DoD:* Endpoint retorna datos consolidados históricos en menos de 200 ms.
- **Tarea 18.2 [Backend] - Consultas de Gestión Operativa (Pedidos e Inventario)**
 - *Descripción:* Escribir endpoints para obtener conteos de pedidos por estado, stock crítico actual de insumos, total de certificados generados frente a pendientes de envío, y auditorías de actividad de usuarios.

- *DoD*: Datos se entregan agrupados en un solo JSON estructurado para el dashboard.
 - **Tarea 18.3 [Backend] - Servicio de Filtros Multidimensionales**
 - *Descripción*: Programar filtros avanzados que admitan combinaciones de búsqueda: por canal de origen, rango de fechas, productos específicos, nombre de campaña, ubicación geográfica y tipo de cliente.
 - *DoD*: Filtrado compuesto en base de datos retorna resultados exactos sin degradar el rendimiento.
 - **Tarea 18.4 [Backend] - Query de Productos con Mayor Rotación**
 - *Descripción*: Escribir consulta SQL que ordene los productos terminados e insumos por mayor cantidad vendida/consumida en un periodo dado.
 - *DoD*: Generación del top 10 de productos más vendidos consumible por la API.
 - **Tarea 18.5 [Frontend] - Componentes de Gráficos de Gestión**
 - *Descripción*: Implementar gráficos interactivos en Angular (líneas, barras horizontales, diagramas de anillo) usando la librería `ngx-charts` conectada a los endpoints analíticos.
 - *DoD*: Gráficas pintan datos dinámicos y reflejan cambios al aplicar filtros.
 - **Tarea 18.6 [Frontend] - Vista de Control y KPIs de Dirección**
 - *Descripción*: Diseñar la pantalla del Dashboard Administrativo organizando tarjetas de indicadores y selectores de rangos de fechas.
 - *DoD*: Dashboard interactivo totalmente funcional y responsivo.
-

SPRINT 19 (Semana 19): Dashboard Público y Visualización Geográfica de Impacto

- **Tarea 19.1 [Backend] - Endpoint Público de Impacto Social**
 - *Descripción*: Desarrollar `GET /api/reports/dashboard-public` que entregue métricas acumuladas (Total donado global, número de donantes por ciudad) sin exponer nombres, documentos, correos o montos individuales de las personas.
 - *DoD*: El endpoint es público (no requiere token JWT) y no expone datos protegidos bajo la ley de Habeas Data.
 - **Tarea 19.2 [Backend] - Servicio de Geolocalización de Donaciones**
 - *Descripción*: Escribir una consulta geoespacial que agrupe las direcciones de la tabla `clients_donors` por comunas, barrios o zonas municipales, retornando coordenadas de coordenadas para pintar en mapas.
 - *DoD*: Datos agregados geográficamente listos en formato GeoJSON.
 - **Tarea 19.3 [Frontend] - Integración de Mapa Interactivo (Leaflet)**
 - *Descripción*: Configurar la librería Leaflet en Angular. Consumir los datos geográficos de donantes agrupados y dibujarlos como burbujas de densidad de impacto en un mapa de la ciudad/región.
 - *Criterio de Aceptación*: El mapa renderiza los puntos geográficos de donaciones acumuladas de forma correcta y visualmente interactiva.
 - **Tarea 19.4 [Frontend] - Landing Page Pública Embebible**
 - *Descripción*: Diseñar el Layout responsivo de la landing de impacto, integrando el mapa Leaflet, tarjetas de indicadores principales e históricos.
 - *DoD*: Vista funciona embebida en un `Iframe` dentro del sitio web principal de la fundación.
-

SPRINT 20 (Semana 20): Exportación Contable, QA y Despliegue Final

- **Tarea 20.1 [Backend] - Generador de Plantilla CSV para World Office**

- *Descripción:* Programar un generador de archivos CSV en la API. Al consultar un periodo de fechas, recolecta pedidos y transacciones, mapeándolos exactamente con las columnas, tipos de datos y códigos de cuenta del importador contable de World Office.
- *DoD:* Descarga de archivo CSV que se importa exitosamente sin errores de estructura en World Office.
- **Tarea 20.2 [DevOps] - Despliegue en Producción y Configuración SSL**
 - *Descripción:* Realizar el build de producción del panel en Angular y desplegar la aplicación y la API Backend en el servidor VPS definitivo. Configurar el dominio HTTPS utilizando Let's Encrypt / Certbot.
 - *DoD:* Portal de administración responde de forma segura en `https://admin.santiagocorazon.org`.
- **Tarea 20.3 [DevOps] - Tareas Programadas de Backup (Cron Jobs)**
 - *Descripción:* Crear e implementar un Cron Job a nivel de sistema operativo para realizar un backup diario automatizado (volcado sql) de la base de datos PostgreSQL y cargarlo en un almacenamiento seguro en la nube.
 - *DoD:* Script de backup configurado y primer archivo de respaldo verificado en el almacenamiento externo.
- **Tarea 20.4 [QA / Gestión] - Pruebas E2E de Aceptación y Cierre**
 - *Descripción:* Ejecutar pruebas completas de aceptación de todos los flujos integrados (Shopify webhook -> Postgres -> Inventario -> PDF -> Correo/WhatsApp). Realizar capacitación grabada al equipo de la fundación.
 - *DoD:* Firma del acta de entrega definitiva y transferencia exitosa de credenciales de producción.